3-6 虚拟存储器

- 1. 分页思想
  - 页与页框
    - **页框/物理块**：主存被划分成固定长度且较小的存储块。
    - **页**：进程被划分成固定长度的程序块。
    - **大小关系**：页大小 = 页框大小。
    - **装入方式**：进程的页可以离散装入主存的页框中。
  - 虚拟地址
    - **虚拟地址/逻辑地址**：程序中使用的地址，不是最终放入主存后的地址。
    - **构成**：`页号 + 页内偏移量`。
    - **页号位数**：决定虚拟页数量。
    - **页内偏移量位数**：决定每页内的存储单元数量。
  - 物理地址
    - **物理地址/主存地址**：程序真正运行时调入主存后使用的地址。
    - **构成**：`页框号 + 页内偏移量`。
    - **页内偏移量不变**：页和页框大小相同，因此虚拟地址和物理地址的页内偏移量位数相同。
  - 页表
    - **作用**：记录一个进程从页号到页框号的映射。
    - **页号隐含**：页大小固定，页表记录所有页对应的页框号，因此页号可由页表项位置隐含。
    - **地址转换**：通过页表查出页框号，再拼接页内偏移量得到物理地址。
  - 分页方式的缺陷
    - **一次性调入问题**：普通分页要求一个进程的所有页面一次性调入主存。
    - **未利用局部性**：不活跃的页也占据页框，浪费主存资源。
    - **虚拟存储器改进**：只把活跃页面调入主存，不活跃页面留在外存。
- 2. 分段思想
  - 段的划分
    - **段**：按程序逻辑把用户进程划分成大小不等的部分。
    - **段内连续**：每个段在主存中的物理地址连续。
    - **段间离散**：不同段之间可以离散放置。
  - 虚拟地址
    - **构成**：`段号 + 段内偏移量`。
    - **段号位数**：决定每个进程最多可有多少段。
    - **段内偏移量位数**：决定单个段的最大大小。
    - **显式提供**：段号和段内偏移量都必须由用户显式提供。
  - 物理地址
    - **不划分字段**：段式系统没有把主存划分成固定单位，因此物理地址不需要按页框字段划分。
    - **地址计算**：知道某段在主存中的基地址和段长，即可推出段内单元的物理地址。
  - 段表
    - **作用**：记录进程逻辑空间与内存空间的映射。
    - **表项内容**：段长、基址等。
    - **段号隐含**：段号顺序递增，可由段表项位置隐含。
    - **定位方式**：基址 + 段内偏移量得到物理地址，同时用段长检查是否越界。
- 3. 段页式思想
  - 基本结构
    - **段表**：每个进程建立一张段表，每个段对应一个段表项。
    - **页表**：每个段表项引出一张页表。
    - **页表作用**：记录该段占用了主存中的哪些页框。
  - 虚拟地址
    - **构成**：`段号 + 段内页号 + 页内地址`。
    - **段号**：定位段表项。
    - **段内页号**：定位该段对应页表中的页表项。
    - **页内地址**：定位页框内的具体存储单元。
  - 物理地址
    - **构成**：`页框号 + 页内地址`。
    - **逻辑关系**：物理地址划分与页式系统相同。
    - **转换路径**：段表找到页表，页表找到页框号，再拼接页内地址。
- 4. 页式虚拟存储器
  - 虚拟存储器思想
    - **核心思想**：把硬盘的一部分空间作为扩展内存使用，对用户虚拟出更大的内存空间。
    - **局部性依据**：与 Cache 类似，利用程序访问的局部性。
    - **运行方式**：进程运行时，只把活跃页面调入内存，不用的页放在外存，需要时再换入。
    - **用户视角**：每个程序都认为自己拥有连续、私有的地址空间。
    - **实际地址**：程序使用的是虚拟地址，操作系统把虚拟地址转换成物理地址。
  - 虚页与实页
    - **虚页**：虚拟地址空间中的页。
    - **实页/页框**：物理地址空间中的页框。
    - **虚拟地址**：`虚页号 + 页内地址`。
    - **物理地址**：`实页号 + 页内地址`。
    - **页内地址相同**：虚页和实页大小相同，转换时页内地址直接保留。
    - **位数关系**：虚页个数通常多于实页个数，因此虚拟地址位数往往比物理地址位数多。
  - 请求页表
    - **虚页号**：对应进程的虚拟页。
    - **实页号**：该虚页在主存中的页框号。
    - **状态位/存在位 P**：表示该页是否已调入内存。
    - **访问字段 A**：记录访问次数或最近未访问时间，供页面置换算法参考。
    - **修改位 M**：标识该页调入内存后是否被修改；若修改过，换出时要写回外存。
    - **外存地址**：指出该页在外存中的位置，供调入时使用。
  - 基本地址转换
    - **页表基址寄存器**：保存当前进程页表的首地址。
    - **查页表**：根据虚页号找到页表项。
    - **页表命中**：若装入位/有效位为 1，取出实页号。
    - **形成物理地址**：`实页号 + 页内地址`。
- 5. TLB 快表
  - 页表缺陷
    - **页表常驻内存**：每次访问都要先访问主存中的页表。
    - **效率问题**：一次数据访问可能先查页表再访问数据，访存次数增加。
    - **改进思路**：模仿 Cache，设计一个存放常用页表项的高速硬件。
  - TLB 概念
    - **快表/TLB**：存放经常访问的虚页页表项的特殊硬件。
    - **作用**：加速虚拟地址到物理地址的转换。
    - **全相联 TLB**：tag 就是虚页号。
    - **组相联 TLB**：tag 是虚页号高位，另有组索引字段。
    - **有效位**：TLB 表项有有效位，tag 相等但有效位为 0 仍属于 TLB 缺失。
  - TLB 命中
    - **判断**：虚页号与 TLB 表项 tag 匹配，且有效位为 1。
    - **结果**：直接取出实页号，与页内地址拼接成物理地址。
    - **重要结论**：TLB 命中，则页表一定命中。
  - TLB 缺失但页表命中
    - **查慢表**：访问主存中的页表。
    - **页表命中**：页表项有效位为 1，说明该页在主存中。
    - **形成地址**：取页表中的实页号，与页内地址拼接得到物理地址。
    - **更新 TLB**：把该页表项写入 TLB，供后续访问加速。
  - TLB 缺失且页表缺失
    - **缺页判断**：页表项有效位为 0，说明目标虚页不在主存。
    - **缺页处理**：把需要的虚页从外存调入主存。
    - **无空闲页框**：根据页面置换算法换出主存中的某页；若被换出页修改过，要写回外存。
    - **有空闲页框**：根据页表中的外存地址字段，把缺失虚页读入空闲页框。
    - **更新表项**：更新页表和 TLB，将有效位置 1，实页号改为新页框号。
    - **异常性质**：缺页处理属于异常，会中断当前指令；处理结束后重新执行本条指令。
    - **重新执行**：由于 TLB 和页表已更新，重新执行时通常回到 TLB 命中的情况。
- 6. CPU 访存全过程
  - 地址转换阶段
    - **CPU 给出虚拟地址。**
    - **查 TLB**：若 TLB 命中，直接转换为物理地址。
    - **TLB 缺失**：访问主存中的页表。
    - **页表命中**：更新 TLB，并把虚拟地址转换为物理地址。
    - **页表缺失**：触发缺页处理，把页从外存调入主存，更新页表和 TLB。
  - Cache 访问阶段
    - **使用物理地址**：地址转换结束后，用得到的物理地址访问 Cache/主存。
    - **Cache 命中**：直接在 Cache 中存取数据。
    - **Cache 缺失**：检查 Cache 是否有空闲行。
    - **有空闲行**：把主存块送入 Cache，并设置 tag 和有效位。
    - **无空闲行**：从 Cache 替换出一块，再装入目标主存块。
  - TLB、页表、Cache 可能性
    - **TLB 命中、页表命中、Cache 命中**：可能。
    - **TLB 命中、页表命中、Cache 缺失**：可能。
    - **TLB 缺失、页表命中、Cache 命中**：可能。
    - **TLB 缺失、页表命中、Cache 缺失**：可能。
    - **TLB 缺失、页表缺失、Cache 缺失**：可能。
    - **TLB 命中、页表缺失、Cache 缺失**：不可能，因为 TLB 命中则页表一定命中。
    - **TLB 命中、页表缺失、Cache 命中**：不可能，因为 TLB 命中则页表一定命中。
    - **TLB 缺失、页表缺失、Cache 命中**：不可能，因为页表缺失表示信息不在主存，也一定不在 Cache 中。
  - 命中定义
    - **TLB 命中**：TLB 中有该表项，且有效位为 1。
    - **页表命中**：页表中该表项有效位为 1。
    - **有效位为 0**：即使存在表项，也不属于命中。
- 7. 段式虚拟存储器
  - 地址转换
    - **虚拟地址**：`段号 + 段内地址`。
    - **段表基址寄存器**：保存段表首地址。
    - **段表项**：包含装入位、段首址、段长等信息。
    - **越界检查**：段内地址需要与段长比较。
    - **形成物理地址**：若段已装入且未越界，段首址 + 段内地址得到物理地址。
- 8. 段页式虚拟存储器
  - 地址转换
    - **虚拟地址**：`段号 + 段内页号 + 页内地址`。
    - **段表查询**：由段号查段表，得到页表始址和页表大小。
    - **页表查询**：由段内页号查该段的页表，得到实页号。
    - **形成物理地址**：`实页号 + 页内地址`。
    - **装入位检查**：段表和页表中都可能需要检查装入位/有效位。

by joshuji 由 幕布 发布